

UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Course Name: Operating System Lab

Course Code: CSE 402

LAB REPORT

Lab Report Number : 06

Lab Report Name : Priority Scheduling non preemptive & preemptive Scheduling Algorithm

Date of Submission : 30.08.2026

Submitted By:

Name : Ejaj Ahmmod Lemon

ID : 23101160

Section: C2

Submitted To:

Atia Rahman Orthi

Lecturer

University of Asia Pacific

1. Problem Statement:

We are to implement **Priority CPU Scheduling** in two different ways: **Non-Preemptive** and **Preemptive** Priority Scheduling. Each process is defined by its **Process ID (PID), Arrival Time (AT), Burst Time (BT), and Priority (PR)**, where a **smaller priority number indicates a higher priority**.

For **Non-Preemptive Priority Scheduling**, once a process starts execution, it continues until its CPU burst is completed. At each scheduling point, the scheduler selects the arrived and incomplete process having the highest priority.

For **Preemptive Priority Scheduling**, the currently running process can be interrupted if a newly arrived process has a higher priority than the currently running process.

The program should generate the **Gantt Chart** and calculate the following scheduling parameters for each process:

- **Completion Time (CT)**
- **Turnaround Time (TAT = CT − AT)**
- **Waiting Time (WT = TAT − BT)**
- **Average Turnaround Time**
- **Average Waiting Time**

The scheduler must also handle **CPU idle periods** when no process has arrived at the current time. If two or more processes have the same priority, the process with the **earlier arrival time** is selected first, followed by **lexicographical order of PID** if necessary.

Given Input

For the output shown in the figure, the processes are:

PID	AT	BT	Priority
P1	0	3	3
P2	1	4	2
P3	2	6	4
P4	3	4	6
P5	5	2	10

Here, **lower priority value means higher priority**.

Expected Output

The program should display the **Gantt Chart**, individual process scheduling information (**CT, TAT, WT**), and the **average TAT and WT** for both:

1. **Priority Scheduling – Non-Preemptive**
2. **Priority Scheduling – Preemptive**

For the given data, the output shows:

- **Non-Preemptive:** Average TAT = **9.6**, Average WT = **5.8**
- **Preemptive:** Average TAT = **10.0**, Average WT = **6.2**

2. Objectives:

The main objectives of implementing the Priority Scheduling non preemptive & preemptive Scheduling Algorithm are:

- Understand and implement Priority CPU Scheduling in both Non-Preemptive and Preemptive modes.
- Select processes based on priority, where a lower numerical value represents a higher priority.
- Handle process arrival times correctly and manage CPU idle periods when no process is available.
- Apply deterministic tie-breaking using arrival time and PID to ensure consistent process ordering.
- Generate the Gantt Chart to visualize the execution order and CPU scheduling timeline.
- Calculate per-process scheduling metrics, including Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).
- Calculate the Average Turnaround Time and Average Waiting Time to evaluate scheduling performance.
- Compare Preemptive and Non-Preemptive Priority Scheduling in terms of waiting time, turnaround time, fairness, and CPU efficiency.
- Analyze the possibility of starvation for low-priority processes in priority-based scheduling.

3. Source Code Screenshot:

```python
processes = [
    {"pid": "P1", "at": 0, "priority": 3, "bt": 3},
    {"pid": "P2", "at": 1, "priority": 2, "bt": 4},
    {"pid": "P3", "at": 2, "priority": 4, "bt": 6},
    {"pid": "P4", "at": 3, "priority": 6, "bt": 4},
    {"pid": "P5", "at": 5, "priority": 10, "bt": 2}
]

def non_preemptive(processes):
    p = [x.copy() for x in processes]
    time = 0
    completed = 0
    n = len(p)
    result = []
    gantt = []
    for x in p:
        x["completed"] = False

    while completed < n:
        available = []
        for x in p:
            if x["at"] <= time and x["completed"] == False:
                available.append(x)
        if not available:
            time += 1
            continue
        available.sort(key=lambda x: (x["priority"], x["at"]))
        current = available[0]
        pid = current["pid"]
        at = current["at"]
        priority = current["priority"]
        bt = current["bt"]
        start = time
        time += bt
        ct = time
        tat = ct - at
        wt = tat - bt
        current["completed"] = True
        result.append([
            pid, at, bt, priority, ct, tat, wt
        ])
        gantt.append([pid, start, ct])
        completed += 1
    return result, gantt

def preemptive(processes):
    p = [x.copy() for x in processes]
    n = len(p)
    time = 0
    completed = 0
    for x in p:
        x["rt"] = x["bt"]
        x["ct"] = 0
    gantt = []

    while completed < n:
        available = []
        for x in p:
            if x["at"] <= time and x["rt"] > 0:
                available.append(x)
        if not available:
            time += 1
            continue
        available.sort(key=lambda x: (x["priority"], x["at"]))
        current = available[0]
        pid = current["pid"]
        if len(gantt) == 0 or gantt[-1][0] != pid:
            gantt.append([pid, time, time + 1])
        else:
            gantt[-1][2] = time + 1
        current["rt"] -= 1
        time += 1

        if current["rt"] == 0:
            current["ct"] = time
            completed += 1
    result = []
    for x in p:
        pid = x["pid"]
        at = x["at"]
        bt = x["bt"]
        priority = x["priority"]
        ct = x["ct"]
        tat = ct - at
        wt = tat - bt
        result.append([
            pid, at, bt, priority, ct, tat, wt
        ])
    return result, gantt


def display(title, result, gantt):
    print("\n" + "=" * 65)
    print(title)
    print("=" * 65)
    print("\nGantt Chart:")
    for g in gantt:
        print(f"| {g[0]} ", end="")
    print("|")
    print(gantt[0][1], end="")
    for g in gantt:
        print("\nAverage TAT =", round(avg_tat, 2))
    print("Average WT  =", round(avg_wt, 2))
    return avg_tat, avg_wt
non_result, non_gantt = non_preemptive(processes)
pre_result, pre_gantt = preemptive(processes)
non_tat, non_wt = display(
    "PRIORITY SCHEDULING - NON-PREEMPTIVE",
    non_result,
    non_gantt
)

pre_tat, pre_wt = display(
    "PRIORITY SCHEDULING - PREEMPTIVE",
    pre_result,
    pre_gantt
)

print("\n" + "=" * 65)
print("PERFORMANCE COMPARISON")
print("=" * 65)

print("Non-Preemptive Average WT :", round(non_wt, 2))
print("Preemptive Average WT     :", round(pre_wt, 2))

print("Non-Preemptive Average TAT:", round(non_tat, 2))
print("Preemptive Average TAT    :", round(pre_tat, 2))

if pre_wt < non_wt:
    print("\nPreemptive has better performance than Non-Preemptive")
elif non_wt < pre_wt:
    print("\nNon-Preemptive has better performance than Preemptive")
else:
    print("\nBoth have the same performance")
```

Figure 1: Source Code of the Priority Scheduling non preemptive & preemptive Scheduling Algorithm

4. Output Screenshot:

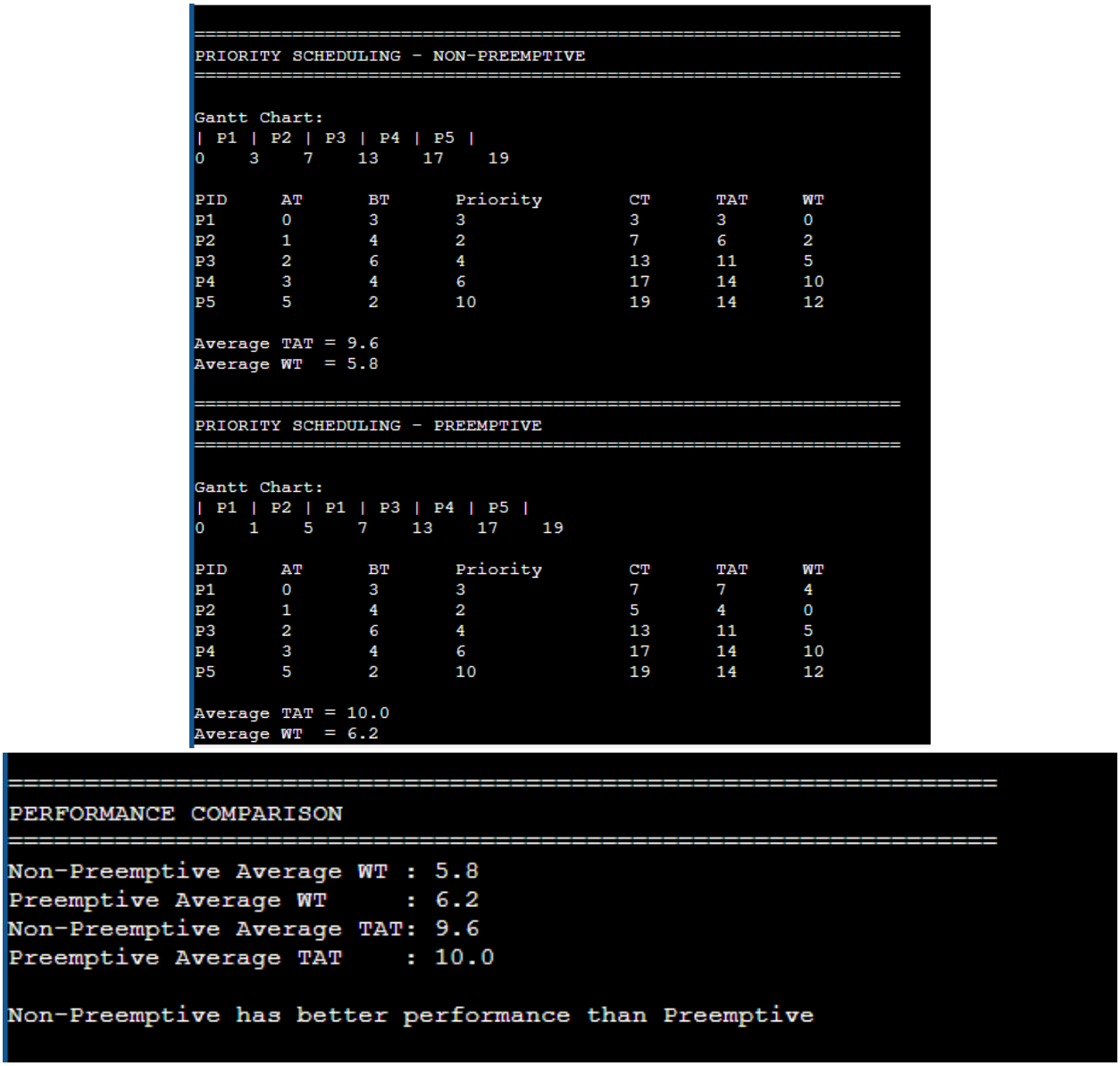


Figure 2: Execution Output of the **Priority Scheduling non preemptive & preemptive Scheduling Algorithm**

The program successfully executes the **Priority CPU Scheduling Algorithm** in both **Non-Preemptive and Preemptive modes** for the given processes. It displays the **Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)** for each process, along with the **Average Turnaround Time and Average Waiting Time**. The Gantt Charts show the execution sequence of the processes according to their assigned priority, where a lower priority value

indicates a higher priority. The performance comparison shows that **Non-Preemptive Priority Scheduling performs better than Preemptive Priority Scheduling**, with an **Average WT of 5.8** and **Average TAT of 9.6**, compared to **6.2** and **10.0** respectively for Preemptive Priority Scheduling.

5. Discussion:

The implementation successfully demonstrates the working of **Priority CPU Scheduling** using both **Non-Preemptive and Preemptive** approaches. In this scheduling method, processes with a **lower priority number are given higher priority**. The program correctly considers the arrival time and priority of each process and generates the corresponding Gantt Chart.

From the performance comparison, **Non-Preemptive Priority Scheduling** achieved an average Waiting Time of **5.8** and an average Turnaround Time of **9.6**, while **Preemptive Priority Scheduling** resulted in an average Waiting Time of **6.2** and an average Turnaround Time of **10.0**. Therefore, for the given set of processes, the non-preemptive approach provides slightly better performance.

The result also shows that preemption does not always improve scheduling performance. Although preemptive scheduling can respond more quickly to newly arrived high-priority processes, the interruption of the currently running process can increase waiting and turnaround times. On the other hand, non-preemptive scheduling allows a selected process to complete without interruption, resulting in lower average waiting and turnaround times for this particular input.

However, the performance may vary depending on the **arrival time, burst time, and priority** of the processes. Priority scheduling can also cause **starvation**, where low-priority processes may have to wait for a long time if higher-priority processes continue to arrive. Overall, the experiment provides a clear understanding of how priority affects CPU allocation and demonstrates that **Non-Preemptive Priority Scheduling performed better than Preemptive Priority Scheduling for the given test case**.

6. Conclusion:

The experiment successfully implemented and evaluated **Priority CPU Scheduling** using both **Non-Preemptive** and **Preemptive** approaches. The scheduler assigns CPU time according to process priority, where a **smaller priority value represents a higher priority**. The program also correctly handles different arrival times, generates the Gantt Chart, and calculates **Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT)** for each process.

For the given set of processes, the performance results are:

Scheduling Method	Average WT	Average TAT
Non-Preemptive Priority	**5.8**	**9.6**
Preemptive Priority	6.2	10.0

Comparison

The results clearly show that **Non-Preemptive Priority Scheduling performs better for this particular input** because it has both a lower average Waiting Time and a lower average Turnaround Time.

- **Waiting Time:** Non-Preemptive = **5.8**, Preemptive = **6.2**
- **Turnaround Time:** Non-Preemptive = **9.6**, Preemptive = **10.0**

Thus, the non-preemptive algorithm reduces the average waiting time by **0.4 time units** and the average turnaround time by **0.4 time units** compared with the preemptive algorithm.

For the given test case, Non-Preemptive Priority Scheduling is better. It provides better overall performance because processes that are selected for execution continue until completion, avoiding interruptions and the additional scheduling overhead associated with preemption. This results in lower average waiting and turnaround times in this particular scenario.

However, **this does not mean Non-Preemptive Priority Scheduling is always better**. The choice depends on the requirements of the system.

Non-Preemptive Priority Scheduling is more suitable when:

- Processes do not require immediate response.
- Lower scheduling overhead is preferred.
- Once a high-priority process starts, completing it without interruption is beneficial.
- The workload is relatively stable.

Preemptive Priority Scheduling is more suitable when:

- Fast response to newly arriving high-priority processes is important.
- The system is interactive or real-time.
- A newly arrived critical process should immediately receive CPU time.
- Responsiveness is more important than minimizing average waiting time.

One important disadvantage of both priority-based approaches is **starvation**. A low-priority process may wait for a long time if higher-priority processes continuously arrive. This problem can be reduced by techniques such as **aging**, where the priority of a waiting process gradually increases.

Overall Conclusion

In conclusion, the experiment demonstrates that **there is no universally best priority scheduling algorithm**; performance depends on the characteristics of the workload. Based on the experimental results, **Non-Preemptive Priority Scheduling is the better choice for the given process set**, achieving **Average WT = 5.8** and **Average TAT = 9.6**, compared with **6.2** and **10.0** for the preemptive approach. Nevertheless, **Preemptive Priority Scheduling can be preferable in systems where responsiveness and immediate execution of high-priority processes are more important**. Therefore, the selection between the two should be based on the system's performance requirements, workload, and response-time needs.